

UniApp showLoading与showModal实战示例大全

在UniApp开发中，`uni.showLoading()` 和 `uni.showModal()` 是高频使用的交互API。前者用于展示加载状态，告知用户当前处于等待过程（如接口请求中）；后者用于弹出确认弹窗，获取用户明确操作（如删除确认、权限申请）。本文通过多个实战场景，详解两者的参数配置、使用技巧及避坑要点，助力开发者快速掌握并灵活运用。

一、showLoading：加载状态提示的核心用法

`showLoading`用于展示带有加载图标的提示框，默认不会自动关闭，需手动调用 `uni.hideLoading()` 关闭，核心特点是阻断用户操作（可通过`mask`参数控制），适用于接口请求、数据处理等耗时操作场景。

1. 核心参数说明

参数名	类型	可选值/说明	默认值
title	String	加载提示文本，如“加载中...”	无
mask	Boolean	true（显示遮罩，禁止点击）/false（不禁止点击）	false
success	Function	接口调用成功的回调函数	无
fail	Function	接口调用失败的回调函数	无

2. 实战场景示例

2.1 基础场景：接口请求加载提示

最常用场景：发起接口请求前显示加载提示，请求完成（成功/失败）后关闭提示，形成完整交互闭环。

Code block

```
1  <template>
2    <view class="data-container">
3      <button @click="fetchData">获取用户数据</button>
4      <view v-if="userInfo" class="user-info">
5        用户名: {{ userInfo.name }}<br/>
6        手机号: {{ userInfo.phone }}
7      </view>
8    </view>
9  </template>
10
11  <script setup>
12  import { ref } from 'vue'
13
14  const userInfo = ref(null)
15
16  // 发起接口请求并展示加载提示
17  const fetchData = async () => {
18    try {
19      // 1. 发起请求前显示加载提示, 开启遮罩禁止点击
20      uni.showLoading({
21        title: '加载中...',
22        mask: true // 关键: 防止用户重复点击
23      })
24
25      // 2. 模拟接口请求 (实际开发替换为真实接口)
26      const res = await uni.request({
27        url: 'https://api.example.com/user/info',
28        method: 'GET',
29        data: { userId: 123 }
30      })
31
32      // 3. 请求成功, 处理数据
33      if (res.statusCode === 200 && res.data.code === 0) {
34        userInfo.value = res.data.data
35        // 可选: 加载成功后可配合showToast提示结果
36        uni.showToast({ title: '加载成功', icon: 'success' })
37      } else {
38        uni.showToast({ title: '数据获取失败', icon: 'error' })
39      }
40    } catch (err) {
41      // 4. 请求失败, 提示错误信息
42      uni.showToast({ title: '网络异常, 请重试', icon: 'error' })
43      console.error('接口请求失败: ', err)
44    } finally {
45      // 5. 无论成功失败, 都关闭加载提示 (核心: 避免加载框一直显示)
46      uni.hideLoading()
47    }
```

```
48 }
49 </script>
50
51 <style scoped>
52 .data-container {
53   padding: 20rpx;
54 }
55 .user-info {
56   margin-top: 30rpx;
57   padding: 20rpx;
58   background-color: #f5f5f5;
59   border-radius: 10rpx;
60 }
61 </style>
```



核心原则： `uni.hideLoading()` 必须在 `finally` 中调用，确保无论请求成功或失败，加载提示都会关闭，避免出现“加载框卡死”的问题。

2.2 进阶场景：加载超时处理

针对网络较差的场景，设置加载超时时间，超时后自动关闭加载提示并提示用户，提升体验。

Code block

```
1  const fetchDataWithTimeout = async () => {
2    // 定义超时时间 (5秒)
3    const timeout = 5000
4    let timeoutTimer = null
5
6    try {
7      uni.showLoading({ title: '加载中...', mask: true })
8
9      // 1. 设置超时定时器
10     timeoutTimer = setTimeout(() => {
11       throw new Error('加载超时，请检查网络')
12     }, timeout)
13
14     // 2. 发起接口请求
15     const res = await uni.request({
16       url: 'https://api.example.com/timeout-test',
17       method: 'GET'
18     })
19
```

```
20 // 3. 请求成功，清除超时定时器
21 clearTimeout(timeoutTimer)
22 uni.hideLoading()
23 uni.showToast({ title: '加载成功', icon: 'success' })
24 } catch (err) {
25 // 4. 超时或请求失败，处理异常
26 clearTimeout(timeoutTimer) // 清除定时器
27 uni.hideLoading()
28 uni.showToast({ title: err.message, icon: 'error' })
29 }
30 }
```

二、showModal：模态框交互的实战运用

showModal用于弹出带有确认/取消按钮的模态对话框，需用户明确操作后才能继续，适用于删除确认、权限申请、重要信息提示等场景，核心特点是“强制交互”，确保用户感知关键操作。

1. 核心参数说明

参数名	类型	可选值/说明	默认值
title	String	模态框标题	无
content	String	模态框内容，可换行（\n）	无
showCancel	Boolean	true（显示取消按钮）/false（仅显示确认）	true
cancelText	String	取消按钮文本	"取消"
confirmText	String	确认按钮文本	"确定"
cancelColor	String	取消按钮颜色（十六进制）	#000000
confirmColor	String	确认按钮颜色（十六进制）	#52C41A
success	Function	用户点击按钮后的回调	无

2. 实战场景示例

2.1 场景1：删除确认弹窗（高危操作）

删除数据属于高危操作，需通过模态框明确提示风险，防止用户误操作，确认后再执行删除逻辑。

Code block

```
1  <template>
2    <view class="item-container">
3      <view class="item">2024年旅行计划.docx</view>
4      <button @click="showDeleteModal" class="delete-btn">删除</button>
5    </view>
6  </template>
7
8  <script setup>
9    // 显示删除确认模态框
10   const showDeleteModal = () => {
11     uni.showModal({
12       title: '确认删除',
13       content: '此文件删除后将无法恢复，是否确认删除？', // 明确风险提示
14       confirmText: '删除', // 按钮文案与操作一致
15       confirmColor: '#FF4D4F', // 红色突出风险
16       cancelColor: '#666666',
17       success: (res) => {
18         // 用户点击确认按钮
19         if (res.confirm) {
20           // 执行删除逻辑（接口请求或本地数据删除）
21           deleteFile()
22         }
23         // 用户点击取消按钮，无需额外操作（可按需处理）
24         else if (res.cancel) {
25           console.log('用户取消删除')
26         }
27       }
28     })
29   }
30
31   // 模拟删除文件的接口请求
32   const deleteFile = async () => {
33     try {
34       uni.showLoading({ title: '删除中...', mask: true })
35       // 实际开发替换为真实删除接口
36       await uni.request({
37         url: 'https://api.example.com/file/delete',
38         method: 'POST',
39         data: { fileId: 456 }
40       })
41       uni.hideLoading()
42       uni.showToast({ title: '删除成功', icon: 'success' })
43       // 后续：刷新文件列表
```

```

44     } catch (err) {
45         uni.hideLoading()
46         uni.showToast({ title: '删除失败', icon: 'error' })
47     }
48 }
49 </script>
50
51 <style scoped>
52 .item-container {
53     display: flex;
54     justify-content: space-between;
55     align-items: center;
56     padding: 20rpx;
57     border-bottom: 1px solid #eee;
58 }
59 .delete-btn {
60     color: #FF4D4F;
61 }
62 </style>

```

2.2 场景2：权限申请弹窗（引导用户授权）

当需要获取用户地理位置、相机等权限时，先通过模态框说明权限用途，再调用权限申请API，提升用户授权意愿。

Code block

```

1  // 引导用户授权地理位置权限
2  const requestLocationPermission = () => {
3      uni.showModal({
4          title: '需要地理位置权限',
5          content: '为了为您推荐附近的门店，需要获取您的地理位置权限，是否允许？', // 说明权限用途
6          confirmText: '去授权',
7          confirmColor: '#1890FF',
8          success: async (res) => {
9              if (res.confirm) {
10                 // 调用UniApp权限申请API
11                 const authRes = await uni.getSetting({
12                     scope: 'scope.userLocation' // 地理位置权限标识
13                 })
14
15                 // 已授权：直接获取地理位置
16                 if (authRes.authSetting['scope.userLocation']) {
17                     getLocation()
18                 }
19                 // 未授权：引导用户前往设置页授权

```

```

20         else {
21             uni.openSetting({
22                 success: (settingRes) => {
23                     if (settingRes.authSetting['scope.userLocation']) {
24                         getLocation()
25                     } else {
26                         uni.showToast({ title: '未授权，无法获取位置', icon: 'none' })
27                     }
28                 }
29             })
30         }
31     }
32 }
33 })
34 }
35
36 // 获取用户地理位置
37 const getLocation = () => {
38     uni.getLocation({
39         type: 'gcj02',
40         success: (res) => {
41             console.log('纬度: ', res.latitude, '经度: ', res.longitude)
42             // 后续：根据地理位置推荐门店
43         },
44         fail: () => {
45             uni.showToast({ title: '获取位置失败', icon: 'error' })
46         }
47     })
48 }

```

2.3 场景3：信息提示弹窗（无取消按钮）

当需要用户必须查看信息（如版本更新说明）时，设置 `showCancel: false`，仅保留确认按钮，强制用户阅读后操作。

Code block

```

1  // 版本更新提示弹窗
2  const showUpdateModal = () => {
3      uni.showModal({
4          title: '发现新版本V2.1.0',
5          content: '1. 新增门店预约功能\n2. 优化支付流程，提升安全性\n3. 修复已知bug', //
            换行展示更新内容
6          showCancel: false, // 无取消按钮，必须确认
7          confirmText: '立即更新',
8          confirmColor: '#1890FF',

```

```
9      success: (res) => {
10        if (res.confirm) {
11          // 跳转到应用商店或执行更新逻辑
12          uni.navigateTo({ url: '/pages/update/update' })
13        }
14      }
15    })
16  }
```

三、两者结合使用：完整交互流程示例

实际开发中，showLoading和showModal常结合使用，形成“确认操作→加载处理→结果反馈”的完整交互链，以“批量删除文件”为例：

Code block

```
1  <template>
2    <button @click="batchDelete">批量删除选中文件（3个） </button>
3  </template>
4
5  <script setup>
6  const batchDelete = () => {
7    // 第一步：弹出确认模态框
8    uni.showModal({
9      title: '批量删除确认',
10     content: '您即将删除选中的3个文件，删除后不可恢复，是否确认？',
11     confirmText: '确认删除',
12     confirmColor: '#FF4D4F',
13     success: async (modalRes) => {
14       // 第二步：用户确认后，显示加载提示
15       if (modalRes.confirm) {
16         try {
17           uni.showLoading({ title: '删除中...', mask: true })
18           // 第三步：执行批量删除接口请求
19           const res = await uni.request({
20             url: 'https://api.example.com/file/batch-delete',
21             method: 'POST',
22             data: { fileIds: [1,2,3] }
23           })
24           // 第四步：处理结果，关闭加载提示并反馈
25           uni.hideLoading()
26           if (res.data.code === 0) {
27             uni.showToast({ title: '批量删除成功', icon: 'success' })
28             // 刷新文件列表
29           } else {
30             uni.showToast({ title: res.data.msg || '删除失败', icon: 'error' })
```



```
31         }
32     } catch (err) {
33         // 异常处理
34         uni.hideLoading()
35         uni.showToast({ title: '网络异常', icon: 'error' })
36     }
37 }
38 }
39 })
40 }
41 </script>
```

四、共同避坑要点

1. 避免加载/弹窗叠加

问题：前一个加载提示未关闭时，再次调用showLoading；或模态框未关闭时调用其他弹窗，导致界面混乱。

解决：调用前先关闭同类提示：

Code block

```
1  // 安全调用showLoading的封装函数
2  const safeShowLoading = (options) => {
3      uni.hideLoading() // 先关闭已有加载提示
4      uni.showLoading(options)
5  }
6
7  // 安全调用showModal的封装函数
8  const safeShowModal = (options) => {
9      uni.hideModal() // 先关闭已有模态框
10     uni.showModal(options)
11 }
```

2. 多端适配注意事项

- **样式差异**：各平台模态框样式略有不同（如微信小程序标题字体加粗，App端更圆润），核心功能保持一致即可，无需强行统一。
- **内容长度限制**：H5端模态框内容过长会自动滚动，小程序端需控制内容长度，避免超出屏幕。
- **权限差异**：App端调用位置、相机等权限前，需在manifest.json中提前声明权限，否则模态框引导后仍无法授权。

3. 回调函数逻辑清晰

showModal的success回调中，必须区分 `res.confirm` 和 `res.cancel` 的逻辑，避免用户取消操作后仍执行核心逻辑（如删除、支付）。

五、总结

showLoading和showModal是UniApp交互设计的核心API，两者的合理运用能大幅提升用户体验：

- `showLoading`：聚焦“过程提示”，核心是“开启-关闭”成对使用，通过mask防止重复操作，配合finally确保关闭。
- `showModal`：聚焦“确认交互”，核心是明确提示风险/用途，根据场景控制是否显示取消按钮，回调中区分用户操作。
- 结合使用：形成“确认→加载→反馈”的完整交互链，让用户清晰感知操作流程和结果，减少困惑。

实际开发中，建议将两者封装为通用工具函数，统一配置样式和逻辑，提升开发效率并保证项目交互一致性。

（注：文档部分内容可能由 AI 生成）